

Apache Ratis

A High Performance Raft Library

Tsz-Wo Nicholas Sze

2024.6.3



COMMUNITY
OVER CODE



Bratislava, Slovakia. June 3-5, 2024

About Me

Tsz-Wo Nicholas Sze, Ph.D.

- Software Engineer / Amateur Mathematician
- PMC member of Apache Hadoop, Ozone & Ratis
- Fun facts
 - Used Hadoop to create a π computation record (2010)
 - <https://www.bbc.com/news/technology-11313194>
 - Discovered a deterministic primality proving algorithm for Proth numbers (2008)
 - https://en.wikipedia.org/wiki/Proth_prime#Primality_testing

Agenda

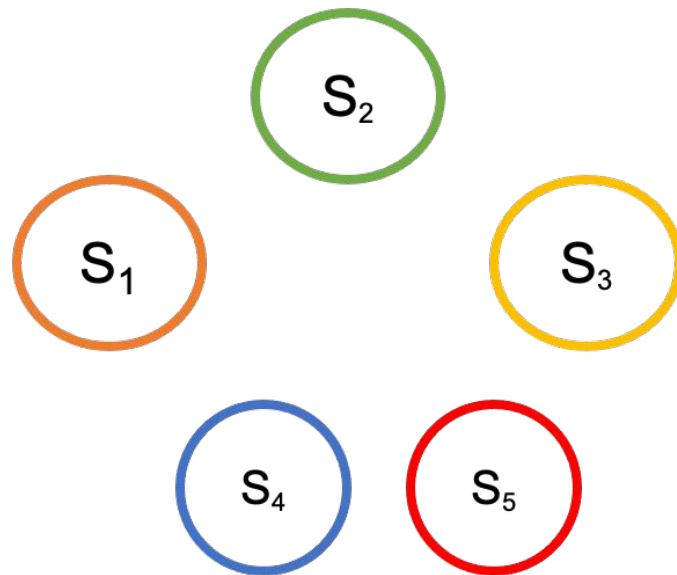
- A Brief Introduction of Raft
- Apache Ratis Community
- Apache Ratis Features

Agenda

- A Brief Introduction of Raft
- Apache Ratis Community
- Apache Ratis Features

Consensus

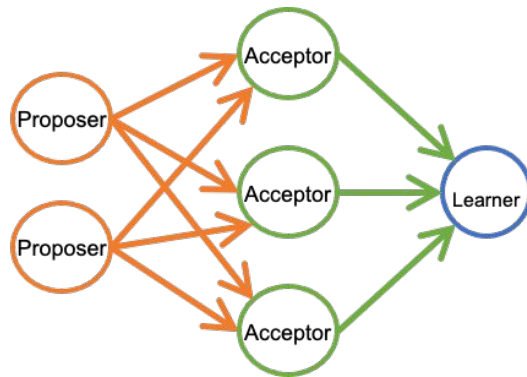
- What is consensus?
 - Multiple servers to **agree a value**
- Typical use cases
 - Log replication
 - Replicated state machines (high availability)



Consensus Algorithms

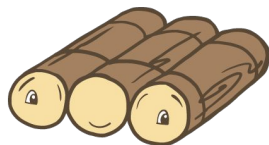
- **Paxos (1990)**

- Works but hard to understand
- Hard to implement (correctly)



- **Raft (2014) – “*In Search of an Understandable Consensus Algorithm*”**

- Easy to understand
- Easy to prove
- Easy to implement



Raft Basic – Leader Election

RPC: requestVote

- Servers are started as **Followers**
- **Randomly timeout** to become a **Candidate** and start a *leader election*
 - Candidate sends *requestVote* to other servers
 - A server votes for a Candidate only if the candidate has a *up-to-date* state.
- The Candidate becomes the **Leader** once it gets a **majority** of the votes
 - Impossible to have two Leaders at the same time.

Raft Basic – Log Replication

RPC: `appendEntries`

- Clients send requests to the Leader
 - A non-leader will reject client requests.
- Leader forwards requests as *log entries* to the Followers via *appendEntries*.
 - Once the Leader has obtained a **majority**, a request is **committed**
 - Committed requests are *safe* regardless of failures or restarts.
- **Heartbeat** to maintain leadership
 - The Leader sends empty `appendEntries` to Followers to when idle.

Raft Library

- **Our Motations (2016)**

- Use Raft in Hadoop Ozone – HA and Write Pipeline

- **“In Search of a Usable Raft Library”**

- A long list of Raft implementations is available
- None of them a general library ready to be consumed by other projects.
- Most of them are tied to another project or a part of another project.

- **We need a Raft library!**

Agenda

- A Brief Introduction of Raft
- Apache Ratis Community
- Apache Ratis Features

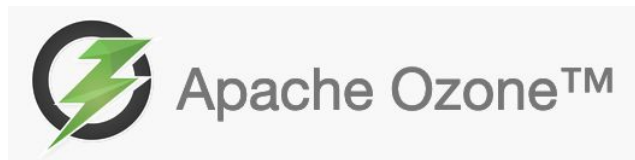
Apache Ratis — A Brief History

- 2016-03: Started at Hortonworks.
- 2017-01: Entered Apache **incubation**.
- 2017-05: Released first version 0.1.0-alpha.
- 2017-04: Hadoop **Ozone** branch started using Ratis!
- 2020-07: Released first **GA** version 1.0.0.
- 2021-02: Became a **top level** Apache project.
- 2021-03: Released the version 2.0.0.
- 2023-11: Released the version 3.0.0.
- 2024-01: Releases the latest version **3.0.1**.



Who uses Apache Ratis?

- **Apache Ozone** (Object Store)
 - SCM/OM high availability
 - Write Pipeline
- **Alluxio** (Data Orchestration for the Cloud)
 - Originally called *Tachyon*
 - Managing all of Alluxio's journaled state
- **Apache IoTDB** (Database for Internet of Things)
 - Replicating application state (HA)
- **Apache Celeborn** (Intermediate Data Service)
 - Replication for high availability



General Ratis Use Case

- You already have a service running on a **single server**
- You want to
 - (1) **replicate** the server data to multiple machines
 - The replication number/cluster membership can be changed in runtime
 - It can tolerate server failures.
 - or
 - (2) have a **HA (highly available)** service
 - When a server fails, another server will automatically take over.
 - Clients automatically failover to the new server.

Apache Ratis is for you!

Apache Ratis — A Raft Library

- Open source, open development, community driven
- Apache License 2.0
- Written in Java 8

Contributions are welcome!

```
static CompletableFuture<?> contributeToApacheRatis(Contributor you) {  
    final CompletableFuture<?> yourFuture =  
        you.submit(yourIdeas -> new PullRequest(you.spendTimeOn(yourIdeas)))  
            .thenAccept(invitation -> new Committer(you, invitation));  
    return yourFuture;  
}
```

Agenda

- A Brief Introduction of Raft
- Apache Ratis Community
- Apache Ratis Features

Apache Ratis — Standard Raft Features

- **Leader Election + Log Replication**

- Automatically elect a leader among the servers in a *Raft group*
- Randomized timeout for avoiding split votes
- Log is replicated in the Raft group

- **Membership Changes**

- Members in a Raft group can be re-configured in runtime.
- Replication factor can be changed in runtime.

- **Log Compaction**

- *Snapshots* are taken periodically
- Send snapshot instead of a long log history.

Apache Ratis – Pluggability

- **Pluggable state machine – the application logic**
 - An application must define its state machine
- **Pluggable metrics**
 - Default is implemented using Dropwizard metrics v4
 - Provided an alternative using Dropwizard metrics v3
 - Applications may provide a custom metrics implementation

Apache Ratis – Pluggability

- **Pluggable RPC**

- Applications may provide their own RPC implementation
- Provided implementations: *gRPC*, Netty, Hadoop RPC



- **Pluggable Raft log**

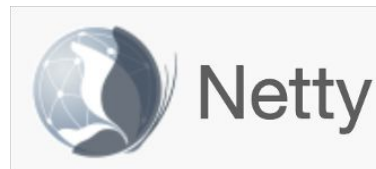
- Users may provide their own log implementation
- The default implementation stores log in *local files*



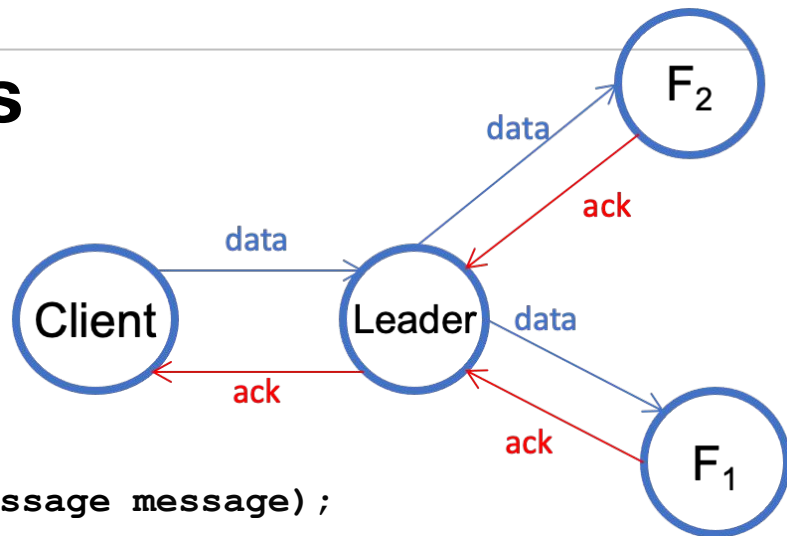
Apache Ratis — High Performance

- **Asynchronous Event Driven Architecture**

- Implemented by
 - gRPC bi-directional stream API
 - Netty asynchronous event-driven network application framework
 - Java CompletableFuture & Executor APIs
- Server-to-server: asynchronous appendEntries
- Client-to-server: asynchronous client requests
 - Also support blocking APIs in client



Apache Ratis – Client APIs



- **Async API**

```
CompletableFuture<RaftClientReply> send(Message message);
```

```
CompletableFuture<RaftClientReply> sendReadOnly(Message message);
```

- **Blocking API** (simply call `CompletableFuture.get()`)

```
RaftClientReply send(Message message) throws IOException;
```

```
RaftClientReply sendReadOnly(Message message) throws IOException;
```

Apache Ratis — High Performance

- **Data Intensive Applications**

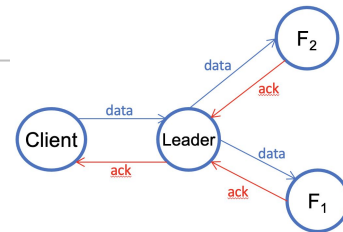
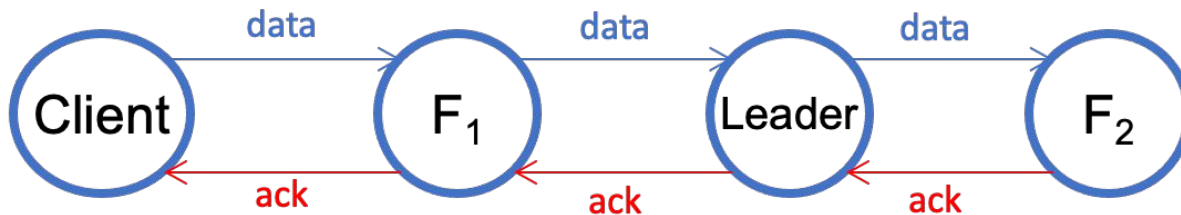
- **In Raft,**

- All transactions and the data are written in the log
 - Log entry: data + metadata
 - State machine: data
- Write amplification
 - A single write may become two or more internal writes

- **In Ratis**

- Application could choose to not written all the data to log
 - Log entry: only metadata
 - State machine: data

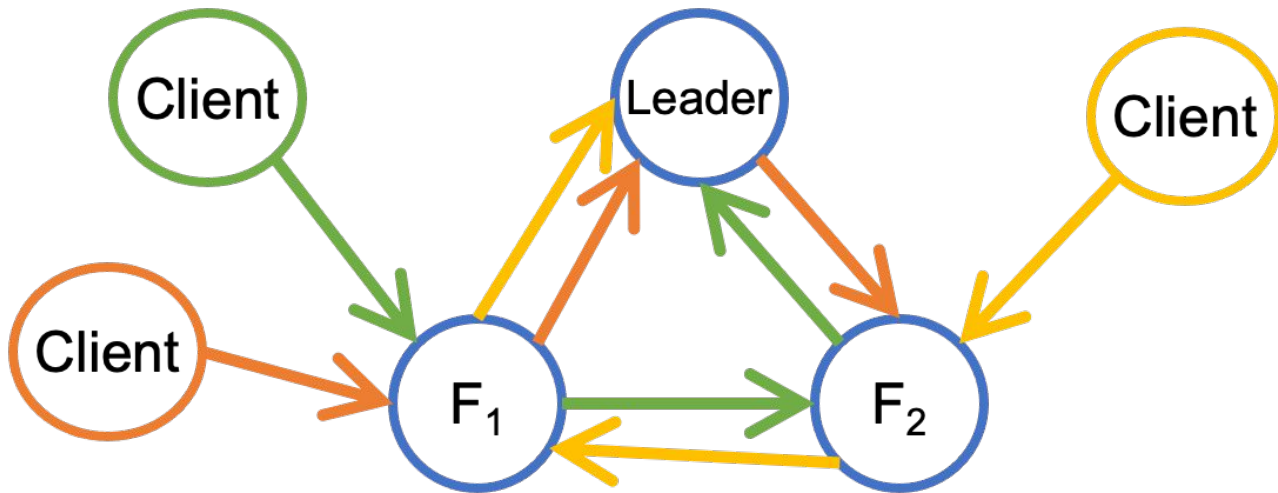
Apache Ratis – Streaming API



- Allow a client writing to **the closest peer**, instead of the Leader
- The first peer forwards the data to other peers
- The client may provide a **routing table** to tell how to forward the data.
- *Netty zero buffer copy* – never create/copy buffers in the code
 - Not necessarily the OS level zero buffer copy

Apache Ratis – Streaming Performance

- The performance can be 3x compared to writing to the Leader
 - Streaming can use the full power of all **three** machines.

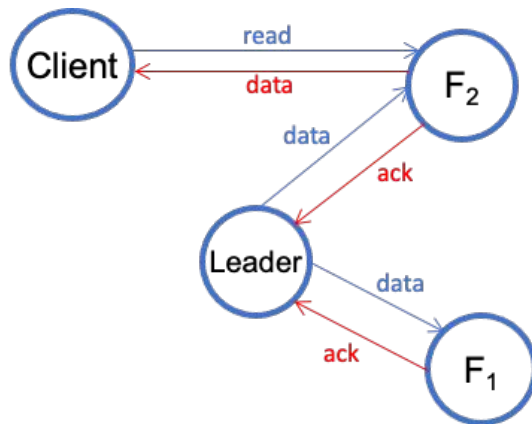


Apache Ratis – Read from Followers

- **Linearizable Read**

- Clients can read updated data from Followers
- Read-index algorithm & Leader lease

- **Stale Read**



```
RaftClientReply sendStaleRead(Message message, long minIndex, RaftPeerId server) throws IOException;
```

```
CompletableFuture<RaftClientReply> sendStaleRead(Message message, long minIndex, RaftPeerId server);
```

- **minIndex** is the minimum log index already committed in the given server
- The returned data can be outdated (stale)

Apache Ratis — Read After Write Consistency

- Suppose $x = 1$, a client sends the following operations **asynchronously**:
 - Write $x = 2$
 - Write $x = 3$
 - Read x
- What will it get for “Read x ”?
 - Intuitively, it *should* get $x = 3$.
 - However, read and write are both **asynchronous** and executed in **different paths**.
 - Although the same client sends all ops, it is like that read and write are sent by two clients.
 - “Read x ” *may* return 1, 2 or 3.
- In Ratis, it support read-after-write consistency (configurable)
 - “Read x ” must return 3.

Apache Ratis – Watch Requests

Client API

- **Watch an index** to satisfy the given **replication level**.

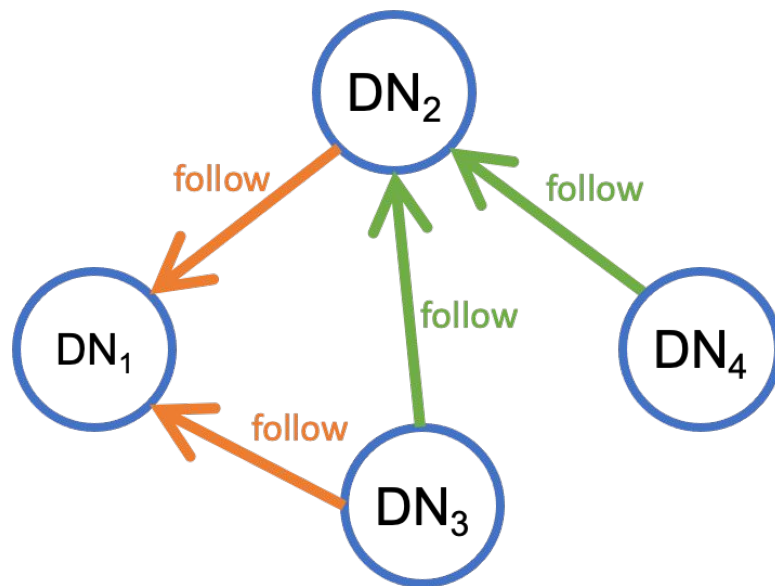
```
RaftClientReply watch(long index, ReplicationLevel replication) throws IOException;
```

```
CompletableFuture<RaftClientReply> watch(long index, ReplicationLevel replication);
```

- **MAJORITY** (default) – committed at the leader and **replicated** to a **majority** of peers (Raft)
- **ALL** – committed at the leader and **replicated** to the **all** peers.
- **MAJORITY_COMMITTED** – **committed** at a **majority** of peers.
- **ALL_COMMITTED** – **committed** at **all** peers.

Apache Ratis — Multi-Raft

- A server can join multiple Raft groups



Apache Ratis — Security

- Support **TLS** connections in *gRPC* and *Ratis Streaming*
 - Applications may pass a Ratis **TlsConf** to build a Ratis client/server
 - The client/server uses the TlsConf to build a Netty **SslContext**
 - The SslContext is used to establish a secure connection to provide
 - Authentication
 - Encryption

Apache Ratis — Leadership

- **Transfer Leadership**

- Change the leader from a server to another server

- **Leader Election Management**

- Pause and resume leader elections

Apache Ratis – Snapshot Management

- **Standard Raft Snapshot**

- Auto-triggering snapshot
- Configurable snapshot retention policy

- **Install Snapshot Notification**

- Notify a follower to install a snapshot from an external source

- **Manual Snapshot Creation**

- API to trigger creating a snapshot

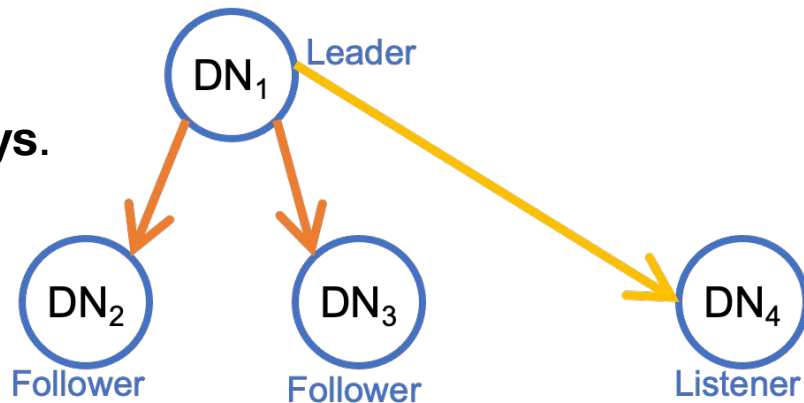
Apache Ratis – Server Priority

- **Priority**

- Servers can be assigned to a priority
- A higher priority server won't vote for a lower priority server
 - unless the higher priority server is *outdated*
- A lower priority server won't become the Leader
 - unless all higher priority servers voted for it

Apache Ratis – Non-voting members

- **Listeners** – for listening log entries
 - They receive appendEntries.
 - They do not vote.
 - They are not counted for majority.
 - They can be served as **hot standbys**.



Apache Ratis – CLI

- **Group:** info, list
- **Peer:** add, remove, setPriority
- **Snapshot:** create
- **Election:** transfer, stepDown, pause, resume
- **Local:** raftMetaConf



COMMUNITY
OVER CODE



Thank you

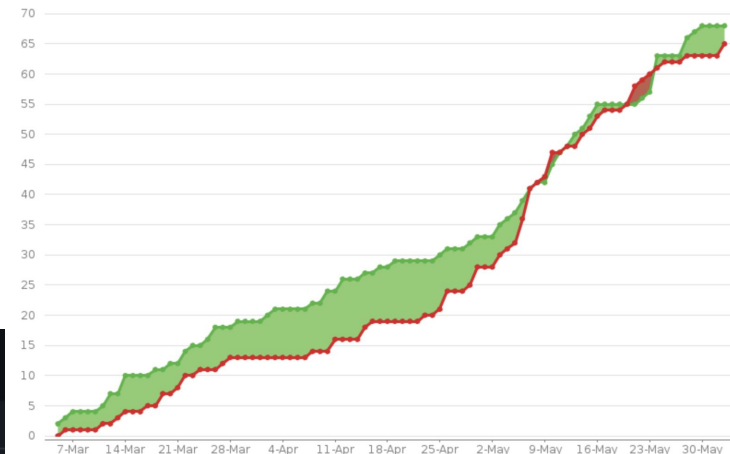
Contributions are welcome!

```
static CompletableFuture<?> contributeToApacheRatis(Contributor you) {  
    final CompletableFuture<?> yourFuture =  
        you.submit(yourIdeas -> new PullRequest(you.spendTimeOn(yourIdeas)))  
            .thenAccept(invitation -> new Committer(you, invitation));  
    return yourFuture;  
}
```

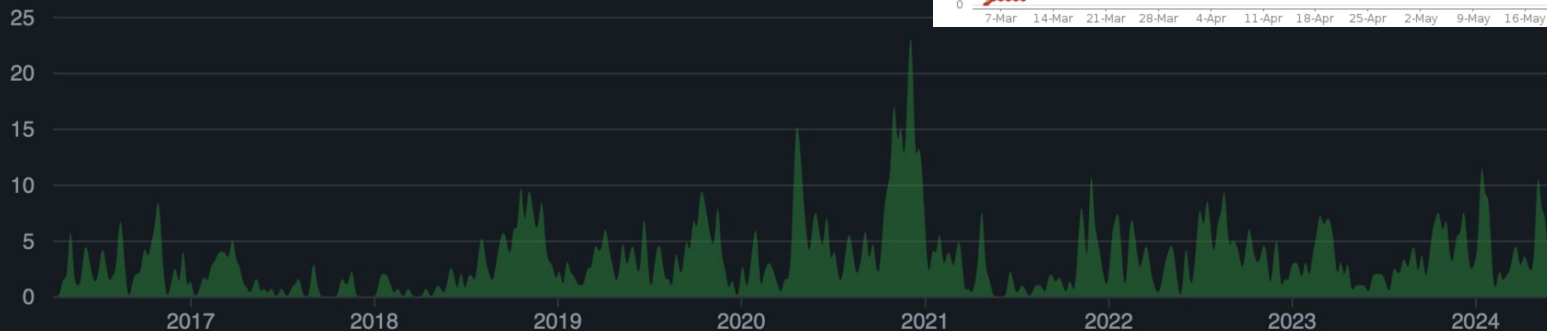
Apache Ratis - Driven by Community

- **93 contributors** (according to github)
- **30 committers**
- **19 PMC members**

This chart shows the number of issues **created** vs. the number of issues **resolved** in the last 90 days.



Contributions to master, excluding merge commits



Apache Ratis - New Releases

- We are currently voting for the **3.1.0** release
 - Thanks **William Song** from *Apache IoTDB* for being the Release Manager
- Ratis Thirdparty Release 1.0.6
 - Vote passed on 2024.5.14
 - Thanks also **Attila Doroszalai** from *Apache Ozone* for being the Release Manager
- Thanks everyone for voting, verifying and testing the releases!



William Song - Thursday, May 30, 2024 11:55:23 AM GMT+2

[VOTE] Apache Ratis Release 3.1.0 rc0

Hi Community,

I'm calling a vote For Apache Ratis Release 3.1.0 rc0.

The git tag to be vote upon:

<https://github.com/apache/ratis/tree/ratis-3.1.0-rc0>

The git commit hash:

0a34940149aef2ed6597564ca1eb5176ccd493c7

The source and binary tarballs can be found at:

<https://dist.apache.org/repos/dist/dev/ratis/3.1.0/rc0/>

Fingerprint of the GPG key release artifacts are signed with:

DCE2 C33D 41C6 2578 969D BAFE 37D6 ECF8 4E78 BC92

My public key to verify signatures can be found in:

<https://dist.apache.org/repos/dist/dev/ratis/KEYS>

Maven artifacts are staged at:

<https://repository.apache.org/content/repositories/orgapacheratis-1145>

This vote will remain open for at least 72 hours.

Please vote on releasing this ratis-3.1.0-rc0. Thanks in advance.

[] +1 approve

[] 0 no opinion

[] -1 disapprove (and reason why)

Starting with my +1(binding)

- Verified checksums, signatures and git hash.

- Checked LICENSE and NOTICE.

- Compared the files in src tarball with the files at the given git tag.

- Built from source.

- Ran regular Ratis CI. [1]

[1] <https://github.com/apache/ratis/actions/runs/9299247901>

Agenda

- A Brief Introduction of Raft
- Apache Ratis Community
- Apache Ratis Features (selected)
- A Short Example

Example: FileStore

- Maintain a file map (**key -> file**)
 - Supported operations: Read, Write, Delete
 - Unsupported operations: List, Rename, etc.
- Asynchronous & In-order
 - Client may submit multiple write requests to
 - Write to multiple files at the same time
 - Each file may have multiple write requests
- File data is managed by the state machine
 - but not store in the raft log
- Performance (depending on hardware)
 - Write throughput: 1.8 GB/s (large files, old results)
 - IOPS: 16,000 txns/s (small files, recent results by **Siyao Meng**)